Services-based Architecture example

1 Giới thiệu

Service-based Architecture (Kiến trúc dựa trên dịch vụ) là một lựa chọn tuyệt vời cho sinh viên vì nó là "người anh em" thực dụng hơn của Microservices. Thay vì chia nhỏ đến mức tối đa, chúng ta nhóm các chức năng có liên quan vào các dịch vụ riêng biệt, giúp hệ thống dễ quản lý nhưng vẫn đảm bảo tính linh hoạt.

Dưới đây là hướng dẫn xây dựng một hệ thống quản lý khóa học đơn giản gồm 3 dịch vụ: **Identity** (Người dùng), **Catalog** (Khóa học), và **Ordering** (Đăng ký).

2 Back-end

2.1 Thiết kế Hệ thống

Hệ thống của chúng ta sẽ hoạt động theo mô hình sau:

- **Identity Service:** Quản lý thông tin giảng viên/sinh viên.
- **Catalog Service:** Danh mục các khóa học hiện có.
- **Ordering Service:** Xử lý việc sinh viên đăng ký vào khóa học.

Cấu trúc cơ sở dữ liệu

Mỗi dịch vụ nên có Database riêng để đảm bảo tính độc lập (Loose Coupling).

2.2 Database Script (SQL Server)

Bạn hãy chạy các script này để tạo 3 database riêng biệt:

```
-- 1. Identity DB
CREATE DATABASE IdentityDB;
GO
USE IdentityDB;
CREATE TABLE Users (Id INT PRIMARY KEY, Name NVARCHAR(100), Email NVARCHAR(100));

-- 2. Catalog DB
CREATE DATABASE CatalogDB;
GO
USE CatalogDB;
CREATE TABLE Courses (Id INT PRIMARY KEY, Title NVARCHAR(200), Price DECIMAL(18,2));

-- 3. Ordering DB
CREATE DATABASE OrderingDB;
GO
USE OrderingDB;
CREATE TABLE Orders (Id INT PRIMARY KEY, UserId INT, CourseId INT, OrderDate DATETIME);
```

2.3 Triển khai C# (ASP.NET Core Web API)

Chúng ta sẽ tạo 3 dự án Web API riêng biệt. Dưới đây là logic cốt lõi cho từng dịch vụ sử dụng **Dapper** (để nhẹ nhàng) hoặc **Entity Framework Core**.

2.3.1 Catalog Service (Dịch vụ Danh mục)

Dịch vụ này trả về danh sách khóa học.

```
// Models/Course.cs
public class Course {
    public int Id { get; set; }
    public string Title { get; set; }
    public decimal Price { get; set; }
}

// Controllers/CoursesController.cs
[ApiController]
[Route("api/[controller]")]
public class CoursesController : ControllerBase {
    [HttpGet]
    public IActionResult GetAll() {
        // Mock data - Trong thực tế sẽ gọi DB
        var courses = new List<Course> {
            new Course { Id = 1, Title = "C# Advanced", Price = 99 },
            new Course { Id = 2, Title = "Microservices Architecture", Price = 150 }
        };
        return Ok(courses);
    }
}
```

2.3.2 Ordering Service (Dịch vụ Đăng ký)

Dịch vụ này cần gọi đến hai dịch vụ kia để kiểm tra thông tin trước khi lưu đơn hàng.

```
[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase {
    private readonly IHttpClientFactory _clientFactory;

    public OrdersController(IHttpClientFactory clientFactory) {
        _clientFactory = clientFactory;
    }

    [HttpPost]
    public async Task<IActionResult> CreateOrder(int userId, int courseId) {
        var client = _clientFactory.CreateClient();

        // 1. Kiểm tra User tồn tại (Gọi Identity Service)
        var userRes = await client.GetAsync($"http://identity-service/api/users/{userId}");

        // 2. Kiểm tra Khóa học tồn tại (Gọi Catalog Service)
        var courseRes = await client.GetAsync($"http://catalog-service/api/courses/{courseId}");
```

```
        if (userRes.IsSuccessStatusCode && courseRes.IsSuccessStatusCode) {
            // 3. Lưu vào OrderingDB
            return Ok(new { Message = "Đăng ký khóa học thành công!" });
        }

        return BadRequest("Thông tin không hợp lệ.");
    }
}
```

2.4 Các bước thực hiện chi tiết

1. **Khởi tạo Solution:** Tạo một Blank Solution trong Visual Studio.
2. **Tạo Project:** Thêm 3 dự án ASP.NET Core Web API.
3. **Cài đặt Thư viện:**
    - Microsoft.EntityFrameworkCore.SqlServer
    - Swashbuckle.AspNetCore (để dùng Swagger thử nghiệm API).
4. **Cấu hình kết nối:** Khai báo ConnectionStrings trong file appsettings.json của từng dịch vụ tương ứng với DB đã tạo ở bước 2.
5. **Chạy ứng dụng:** * Chuột phải vào Solution -> **Configure Startup Projects**.
    - Chọn **Multiple startup projects** và đặt hành động là **Start** cho cả 3 dịch vụ.

2.5 Lưu ý

- **Tính độc lập:** Đừng bao giờ viết một câu lệnh SQL JOIN giữa bảng của IdentityDB và CatalogDB. Nếu cần dữ liệu, hãy gọi qua API.
- **Giao tiếp:** Ở ví dụ trên, chúng ta dùng HTTP (Synchronous). Trong các hệ thống lớn hơn, người ta thường dùng Message Broker (như RabbitMQ) để các dịch vụ nói chuyện với nhau (Asynchronous).

3 Giao diện

Để hoàn thiện hệ thống, chúng ta sẽ xây dựng một giao diện **front-end đơn giản bằng Blazor WebAssembly**. Blazor rất phù hợp cho sinh viên C# vì bạn có thể dùng chung logic (Models) và ngôn ngữ C# mà không cần học sâu về JavaScript.

Giao diện này sẽ kết nối với 3 dịch vụ API mà chúng ta đã thiết kế.

3.1 Cấu trúc Giao diện (Blazor WebAssembly)

Dưới đây là thiết kế giao diện cơ bản bao gồm:

1. **Trang chủ:** Hiển thị danh sách khóa học từ **Catalog Service**.
2. **Nút Đăng ký:** Khi nhấn, sẽ gửi yêu cầu tới **Ordering Service**.

Giao diện mẫu (Mockup)

3.2 Mã nguồn Frontend (C# - Blazor)

A. Model dùng chung

```
// SharedModels/Course.cs
public class Course {
    public int Id { get; set; }
    public string Title { get; set; }
    public decimal Price { get; set; }
}
```

B. Trang Danh sách Khóa học (Courses.razor)

Trang này sẽ gọi API từ CatalogService và hiển thị cho người dùng.

```
@page "/courses"
@inject HttpClient Http

<PageTitle>Danh sách Khóa học</PageTitle>

<div class="container">
    <h3 class="my-4">Khóa học hiện có</h3>

    @if (courses == null) {
        <p><em>Đang tải dữ liệu...</em></p>
    } else {
        <div class="row">
            @foreach (var course in courses) {
                <div class="col-md-4 mb-3">
                    <div class="card shadow-sm">
                        <div class="card-body">
                            <h5 class="card-title">@course.Title</h5>
                            <p class="card-text text-primary">Giá: @course.Price.ToString("C")</p>
                            <button class="btn btn-success" @onclick="() => Enroll(course.Id)">
```

```
                        Đăng ký ngay
                    </button>
                </div>
            </div>
        </div>
    }
    </div>
  }
</div>

@code {
  private List<Course>? courses;
  private int currentUserId = 1; // Giả định sinh viên đã đăng nhập có ID = 1

  protected override async Task OnInitializedAsync() {
    // Gọi đến Catalog Service (Port 5001)
    courses = await Http.GetFromJsonAsync<List<Course>>("https://localhost:5001/api/courses");
  }

  private async Task Enroll(int courseId) {
    // Gọi đến Ordering Service (Port 5003)
    var response = await
Http.PostAsJsonAsync($"https://localhost:5003/api/orders?userId={currentUserId}&courseId={courseId}",
new {});

    if (response.IsSuccessStatusCode) {
      await App.Current.MainPage.DisplayAlert("Thành công", "Bạn đã đăng ký khóa học!", "OK");
    }
  }
}
```

3.3 Database Script bổ sung (Dữ liệu mẫu)

Để giao diện có dữ liệu hiển thị ngay, bạn hãy chạy thêm script chèn dữ liệu này:

```
USE CatalogDB;
INSERT INTO Courses (Id, Title, Price) VALUES
(1, 'Lập trình C# cơ bản', 500000),
(2, 'Kiến trúc Services-based', 750000),
(3, 'Web API nâng cao', 600000);

USE IdentityDB;
INSERT INTO Users (Id, Name, Email) VALUES
(1, 'Sinh Vien A', 'sv_a@school.edu.vn');
```

3.4 Hướng dẫn cấu hình kết nối (CORS)

Vì Frontend và Backend chạy trên các cổng (ports) khác nhau, bạn **bắt buộc** phải bật CORS trong file Program.cs của cả 3 dự án Web API, nếu không trình duyệt sẽ chặn yêu cầu từ giao diện.

Thêm đoạn code này vào Program.cs của các dịch vụ:

```
var builder = WebApplication.CreateBuilder(args);
```

```
// 1. Cấu hình Chính sách CORS
builder.Services.AddCors(options => {
    options.AddPolicy("AllowAll", policy => {
        policy.AllowAnyOrigin().AllowAnyMethod().AllowAnyHeader();
    });
});

var app = builder.Build();

// 2. Sử dụng CORS
app.UseCors("AllowAll");

app.MapControllers();
app.Run();
```

3.5 Các bước kiểm tra

1. Chạy đồng thời cả 3 API Services.
2. Chạy Project Blazor WebAssembly.
3. Mở trình duyệt, truy cập vào trang /courses.
4. Mở **Console (F12)** để theo dõi các luồng gọi API giữa các dịch vụ.

Giao diện này giúp bạn hình dung rõ luồng đi của dữ liệu: **User -> Frontend -> Ordering Service -> (Identity & Catalog Services) -> Databases**.

4 Tích hợp Swagger

Việc tích hợp **Swagger (OpenAPI)** là một bước cực kỳ quan trọng trong kiến trúc Services-based. Nó không chỉ giúp bạn kiểm thử API nhanh chóng mà còn đóng vai trò như một "bản tài liệu sống" để các dịch vụ biết cách giao tiếp với nhau.

Dưới đây là hướng dẫn tích hợp Swagger cho cả 3 dịch vụ và cách quản lý chúng.

4.1 Cài đặt và Cấu hình Swagger (Cho từng API Service)

Mặc dù các template Web API mới của .NET thường có sẵn Swagger, nhưng bạn cần cấu hình chuẩn để hỗ trợ việc gọi liên dịch vụ.

Bước 1: Cài đặt Package

Mở **NuGet Package Manager** và cài đặt cho cả 3 dự án: Swashbuckle.AspNetCore

Bước 2: Cấu hình trong Program.cs

Thêm đoạn code sau vào file Program.cs của **Identity**, **Catalog**, và **Ordering Service**:

```
var builder = WebApplication.CreateBuilder(args);
```

```
// Thêm dịch vụ Swagger vào Container
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(c =>
{
    c.SwaggerDoc("v1", new Microsoft.OpenApi.Models.OpenApiInfo
    {
        Title = "My Service API", // Thay đổi tên theo từng dịch vụ (ví dụ: Identity API)
        Version = "v1",
        Description = "Tài liệu API cho hệ thống quản lý khóa học"
    });
});

var app = builder.Build();

// Kích hoạt Swagger Middleware
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(c =>
    {
        c.SwaggerEndpoint("/swagger/v1/swagger.json", "My Service API V1");
        // Để Swagger là trang mặc định khi chạy (tùy chọn)
        c.RoutePrefix = string.Empty;
    });
}

app.UseAuthorization();
app.MapControllers();
app.Run();
```

4.2 Kiểm thử luồng nghiệp vụ trên Swagger

Khi bạn chạy hệ thống, mỗi dịch vụ sẽ có một trang Swagger riêng (ví dụ: localhost:5001, localhost:5002, localhost:5003).

Luồng kiểm thử chuẩn cho Sinh viên:

1. **Identity Service:** Gọi POST /api/users để tạo một sinh viên mẫu. Copy lấy Id.
2. **Catalog Service:** Gọi GET /api/courses để xem danh sách khóa học. Copy lấy Id khóa học.
3. **Ordering Service:** Truy cập Swagger của dịch vụ này, gọi POST /api/orders và dán 2 ID vừa lấy được vào.

4.3 Lưu ý

- **XML Documentation:** Để Swagger hiển thị các chú thích tiếng Việt cho các tham số, hãy bật "Generate XML documentation file" trong Project Properties và thêm c.IncludeXmlComments(xmlPath); vào cấu hình.
- **Production:** Đảm bảo app.UseSwagger() nằm trong khối if (app.Environment.IsDevelopment()) để tránh lộ cấu trúc API khi triển khai thực tế.

- **Port:** Hãy cố định Port cho từng dịch vụ trong launchSettings.json để Swagger không bị thay đổi địa chỉ mỗi lần khởi động lại.

5 Bài tập

Xây dựng hệ thống **"Quản lý Thư viện Số" (Digital Library System)**.

5.1 Thiết kế Kiến trúc Tổng quát

Hệ thống bao gồm 4 dịch vụ độc lập, mỗi dịch vụ quản lý một vùng dữ liệu riêng (Bounded Context):

1. **Identity Service:** Quản lý tài khoản độc giả và phân quyền.
2. **Book Service:** Quản lý danh mục sách, tác giả và thể loại.
3. **Borrowing Service:** Quản lý việc mượn/trả sách (nghiệp vụ chính).
4. **Notification Service:** Gửi thông báo (giả lập) khi mượn sách thành công hoặc quá hạn.

5.2 Script Database (SQL Server)

Bạn hãy tạo 4 Database riêng biệt để đảm bảo đúng tinh thần của kiến trúc Service-based.

```
-- 1. IdentityDB
CREATE DATABASE IdentityDB;
GO
USE IdentityDB;
CREATE TABLE Users (Id INT PRIMARY KEY, Username NVARCHAR(50), FullName
NVARCHAR(100), Rank NVARCHAR(20));
INSERT INTO Users VALUES (1, 'sv01', 'Nguyen Van A', 'Gold'), (2, 'sv02', 'Tran Thi B', 'Silver');

-- 2. BookDB
CREATE DATABASE BookDB;
GO
USE BookDB;
CREATE TABLE Books (Id INT PRIMARY KEY, Title NVARCHAR(200), Author NVARCHAR(100),
Stock INT);
INSERT INTO Books VALUES (101, 'Clean Code', 'Robert C. Martin', 5), (102, 'Design Patterns', 'Gang of
Four', 2);

-- 3. BorrowingDB
CREATE DATABASE BorrowingDB;
GO
USE BorrowingDB;
CREATE TABLE BorrowRecords (
    Id INT PRIMARY KEY IDENTITY,
    UserId INT,
    BookId INT,
    BorrowDate DATETIME DEFAULT GETDATE(),
    ReturnDate DATETIME NULL
```

```
);

-- 4. NotificationDB
CREATE DATABASE NotificationDB;
GO
USE NotificationDB;
CREATE TABLE Logs (Id INT PRIMARY KEY IDENTITY, Message NVARCHAR(MAX), SentDate
DATETIME DEFAULT GETDATE());
```

5.3 Đề bài chi tiết cho Sinh viên

5.3.1 Nhiệm vụ 1: Xây dựng các API cơ bản

- **Book Service:** Viết API cập nhật số lượng sách trong kho (Stock) sau khi có người mượn.
- **Identity Service:** Viết API kiểm tra hạng của người dùng (Rank). Người dùng hạng "Silver" chỉ được mượn tối đa 3 cuốn.

5.3.2 Nhiệm vụ 2: Xử lý nghiệp vụ liên dịch vụ (Chính)

Tại **Borrowing Service**, sinh viên phải viết API POST /api/borrow:

1. **Check User:** Gọi Identity Service xem User có tồn tại và còn lượt mượn không.
2. **Check Book:** Gọi Book Service xem sách còn trong kho không (Stock > 0).
3. **Transaction:** Nếu 2 bước trên OK, tiến hành lưu vào BorrowingDB.
4. **Update Stock:** Gọi PUT sang Book Service để giảm Stock đi 1.
5. **Notify:** Gọi POST sang Notification Service để ghi log thông báo thành công.

5.3.3 Nhiệm vụ 3: Tích hợp Swagger & Gateway

- Tích hợp Swagger cho cả 4 dịch vụ.
- Sử dụng **Ocelot** để tạo một Gateway duy nhất chạy tại Port 8000.

5.4 Gợi ý cấu hình HttpClient

Trong dự án **Borrowing Service**, sinh viên nên sử dụng `IHttpClientFactory` để gọi các dịch vụ khác:

```
public async Task<bool> CheckBookAvailability(int bookId)
{
    var client = _clientFactory.CreateClient();
    // Gọi sang Book Service (giả sử port 5002)
    var response = await client.GetAsync($"https://localhost:5002/api/books/{bookId}");
    if (response.IsSuccessStatusCode)
    {
        var book = await response.Content.ReadFromJsonAsync<BookDTO>();
        return book.Stock > 0;
    }
    return false;
}
```

5.5 Tiêu chí đánh giá

Tiêu chí	Mô tả
Tính đóng gói	Mỗi service chỉ được truy cập vào Database của chính nó.
Giao tiếp API	Các service gọi nhau qua HTTPClient mượt mà, xử lý được lỗi nếu service kia "chết".
Swagger	Tài liệu API đầy đủ, có mô tả các đầu vào/đầu ra.
Frontend	(Cộng điểm) Có giao diện Blazor liệt kê sách và nhấn nút "Mượn".